

拓扑 AC | CSP-S 线上联测

TopoAC Algorithm Contest

第二轮 第二场

题目名称	路标博弈	果树分区	跃迁中枢	模式警戒
题目类型	传统型	传统型	传统型	传统型
目录	signpost	orchard	hub	pattern
可执行文件名	signpost	orchard	hub	pattern
输入文件名	signpost.in	orchard.in	hub.in	pattern.in
输出文件名	signpost.out	orchard.out	hub.out	pattern.out
每个测试点时限	1.0 秒	1.0 秒	1.0 秒	1.0 秒
内存限制	512 MiB	256 MiB	512 MiB	256 MiB
测试点数目	20	30	40	40
子任务数目	3	5	4	4
测试点是否等分	否	是	否	否

提交源程序文件名

对于 C++ 语言	signpost.cpp	orchard.cpp	hub.cpp	pattern.cpp
-----------	--------------	-------------	---------	-------------

编译选项

对于 C++ 语言	-O2 -std=c++14 -Wall
-----------	----------------------

注意事项（请仔细阅读）

1. 文件名（程序名和输入输出文件名）必须使用英文小写。
2. C++ 中函数 `main()` 的返回值类型必须是 `int`，程序正常结束时的返回值必须是 `0`。
3. 若无特殊说明，结果的比较方式为全文比较（过滤行末空格及文末回车）。
4. 选手提交的程序源文件必须不大于 100KB。
5. 程序可使用的栈空间内存限制与题目的内存限制一致。
6. 禁止在源代码中改变编译器参数（如使用 `#pragma` 命令），禁止使用系统结构相关指令（如内联汇编）和其他可能造成不公平的方法。
7. 请注意所有题目均使用子任务计分，只有通过属于该子任务的全部数据后，才能获得该子任务的分数。

路标博弈 (signpost)

【题目描述】

一张地图由 n 行 m 列的网格组成，格子 (x, y) 上写有非负整数 $a_{x,y}$ 。值为 0 的格子是出口。

从任意起点出发，甲乙两名决策者轮流选择下一步，甲先手。每一步只能走到与当前格四联通且路标值不大于当前值的格子。

甲的首要目标是保证最终到达出口，乙的首要目标是阻止到达出口。但是当乙无法阻止甲到达出口时，甲在能到达出口时希望让路径和尽量小、而乙希望路径和尽量大。两人都知道完整地图并采用最优策略。

路径和包含起点和之后每一步到达的格子；如果同一格被重复访问，每次访问都要重复计入。到达值为 0 的格子后立即结束。

如果乙能够使行走永远不到达出口，或者行走在到达出口前无路可走，则认为从该起点不能保证到达出口，答案为 -1 。起点本身是出口时答案为 0。

请对每个格子分别作为起点，求第一名先手时的博弈结果。

【输入格式】

从 *signpost.in* 读入数据。

第一行包含两个整数 m, n ，依次表示列数和行数。

接下来 n 行，每行包含 m 个整数，描述路标矩阵。

【输出格式】

输出到 *signpost.out* 文件。

输出 n 行，每行 m 个整数。第 x 行第 y 个数表示从 (x, y) 出发的答案。

【样例 1 输入】

```
1 3 3
2 0 1 2
3 3 5 7
4 6 8 4
```

【样例 1 输出】

```
1 0 1 3
2 3 6 10
3 9 16 -1
```

【样例 2 输入】

```
1 5 5
2 0 0 1 2 0
3 0 2 1 0 2
4 0 3 3 3 0
5 0 3 2 1 3
6 3 0 3 8 9
```

【样例 2 输出】

```
1 0 0 1 2 0
2 0 2 1 0 2
3 0 3 5 3 0
4 0 3 -1 -1 3
5 3 0 3 -1 -1
```

【数据范围与子任务】

对于所有数据， $1 \leq n, m \leq 300$ ， $0 \leq a_{x,y} \leq 90000$ 。
本题采用子任务捆绑计分。

子任务	分值	附加限制
1	20	$n, m \leq 10$
2	30	除 0 外的路标值互不相同
3	50	无附加限制

果树分区 (orchard)

【题目描述】

果园由若干棵果树和道路组成。我们把每棵树看作一个节点；任意两棵树之间都能通过道路互相到达，并且任意两个节点之间的简单路径都是唯一的。

果园共有 n 个节点，其中 k 个节点已经结出果实。管理员可以剪断任意条道路，也可以一条都不剪。所有节点都会保留；剪断道路只会改变节点之间的连通关系。

剪断道路后，要求每个连通块都至少包含一个果实节点。一个连通块可以包含多个果实节点。

请最小化所有连通块节点数的最大值，并输出这个最小值。

【输入格式】

从 `orchard.in` 读入数据。

输入仅包含一组数据。

第一行两个正整数 n, k ，分别表示节点数和果实节点数。

接下来 $n - 1$ 行，每行两个整数 u, v ，表示节点 u 与节点 v 之间有一条无向道路。这 $n - 1$ 条边构成一棵简单连通树，节点编号为 1 到 n 。

最后一行包含 k 个两两不同的整数 $a_1, a_2, \dots, a_k$ ，表示果实节点的编号。

【输出格式】

输出到 `orchard.out` 文件。

输出一行一个正整数，表示所有合法剪法中，连通块最大节点数的最小可能值。

【样例 1 输入】

```
1 1 1
2 1
```

【样例 1 输出】

```
1 1
```

【样例 1 解释】

果园只有一个节点，这个节点已经结出果实。无需剪断道路，唯一连通块的大小为 1。

【样例 2 输入】

```
1 9 3
2 1 2
3 1 3
4 2 4
5 2 5
6 3 6
7 3 7
8 7 8
9 7 9
10 4 6 9
```

【样例 2 输出】

```
1 3
```

【样例 2 解释】

剪断道路 $(1, 2)$ 和 $(3, 7)$ 后，三个连通块分别为 $\{1, 3, 6\}$ 、 $\{2, 4, 5\}$ 和 $\{7, 8, 9\}$ ，它们都包含果实节点，大小均为 3。

由于每个连通块都需要至少一个果实节点，三个果实节点至多支持三个连通块。因此不可能把 9 个节点都放入大小不超过 2 的连通块，答案为 3。

【样例 3】

输入与输出见 sample 目录下的 *orchard/orchard3.in* 和 *orchard/orchard3.ans*。

【数据范围】

对于所有测试数据：

- $1 \leq k \leq n \leq 200000$ ；
- $1 \leq u, v \leq n$ ，且所有道路构成一棵简单连通树；
- $1 \leq a_i \leq n$ ，果实节点编号两两不同。

本题采用捆绑测试。只有通过一个子任务的全部测试，才能获得该子任务的分数。五个子任务均无依赖。

子任务	分值	附加约束
1	20	$1 \leq n \leq 20$
2	20	$21 \leq n \leq 2000$
3	20	$2001 \leq n \leq 200000$ 且 $k = 2$
4	20	$2001 \leq n \leq 200000$ 且 $k = 3$
5	20	$2001 \leq n \leq 200000$ 且 $k \notin \{2, 3\}$

【提示】

可以不剪断任何道路。若 $k = 1$ ，剪断任意道路都会产生不含果实的连通块，因此答案为 n ；若 $k = n$ ，可以剪断所有道路，因此答案为 1。

跃迁中枢 (hub)

【题目描述】

一张由 n 行、 m 列房间组成的网格中放置了 k 个跃迁中枢。每个中枢占据一个互不相同的房间，并具有类型 A、B 或 C。

当你位于一个中枢所在的房间时，可以激活它并进行一次单向跃迁：

- A 型：跃迁到与当前房间八方向相邻的一个房间；
- B 型：跃迁到当前行中的任意一个房间；
- C 型：跃迁到当前列中的任意一个房间。

到达一个中枢所在的房间时，该中枢立即被激活；选择起点也视为到达该中枢。因此，即使一个中枢没有可继续到达的中枢，单独选择它作为起点仍会激活它。激活后，你可以停止，也可以进行一次跃迁；如果落点没有中枢，就无法继续跃迁。

你可以任选一个中枢作为起点，并沿一条连续路线反复跃迁。允许重复到达房间或重复激活同一个中枢，但每个中枢只在第一次被激活时计入答案。

求一条路线最多能激活多少个不同的中枢。

【输入格式】

从 *hub.in* 读入数据。

第一行包含三个整数 n, m, k ，分别表示网格的行数、列数和中枢数量。

接下来 k 行，每行包含两个整数 x_i, y_i 和一个大写字母 z_i ，表示第 i 个中枢位于第 x_i 行、第 y_i 列，类型为 z_i 。

保证 $1 \leq x_i \leq n$ 、 $1 \leq y_i \leq m$ ，所有 (x_i, y_i) 两两不同，且 $z_i \in \{A, B, C\}$ 。

【输出格式】

输出到 *hub.out* 文件。

输出一个整数，表示最多能激活的不同中枢数。

【样例 1 输入】

```
1 5 5 6
2 1 1 B
3 1 4 C
4 3 4 A
5 4 5 B
6 4 2 C
7 2 2 A
```

【样例 1 输出】

16

【样例 2 输入】

11 1 1
21 1 A

【样例 2 输出】

11

【样例 3 输入】

13 3 3
21 1 B
31 3 C
42 3 A

【样例 3 输出】

13

【数据范围与子任务】

对于所有数据：

$1 \leq n = m \leq 10^6, \quad 1 \leq k \leq 10^5$

本题采用依赖式子任务捆绑计分。只有一个子任务内的所有测试点均通过，且其依赖的子任务均已得分，才能获得该子任务的全部分数。

子任务	分值	额外限制	依赖
1	15	$k \leq 12$	无
2	25	$k \leq 2000$	1
3	20	不存在 A 型中枢	无
4	40	无额外限制	1、2、3

模式警戒 (pattern)

【题目背景】

一套规则系统使用整数 0 到 $G - 1$ 表示符号。其中 0 和 1 是已经确定的二进制符号，其余符号可以继续展开。系统还保存了一批警戒模式；只要完成后的二进制序列包含其中一个模式，就会触发警报。

【题目描述】

一条生成规则写作

```
1 a -> b1 b2 ... bk
```

它表示：可以选择当前序列中任意一个值为 a 的位置，把该位置替换为 $b_1, b_2, \dots, b_k$ 。同一个符号可能有多条规则，每次展开时可以任选一条。

从单个符号 x 开始，经过有限次替换后，如果序列中的所有符号均为 0 或 1 ，就称得到的二进制序列为 x 的一个完成串。某个符号可能没有任何完成串，也可能有有限或无限多个完成串。

每个警戒模式本身都是一个二进制序列。若某个模式作为连续子串出现在完成串中，就称该完成串触发了警报。

对于每个 $x = 2, 3, \dots, G - 1$ ：

- 如果 x 的每个完成串都会触发至少一个警戒模式，输出 YES；
- 否则输出 NO L ，其中 L 是所有不触发警报的完成串中的最小长度。

特别地，如果 x 没有任何完成串，应输出 YES。

【输入格式】

从 *pattern.in* 读入数据。

第一行包含三个整数 G, N, M ，分别表示符号总数、生成规则数和警戒模式数。

接下来 N 行，每行先给出两个整数 a, k ，随后给出 k 个整数 $b_1, b_2, \dots, b_k$ ，表示规则 $a \rightarrow b_1 b_2 \dots b_k$ 。

接下来 M 行，每行先给出模式长度 l ，随后给出 l 个整数 $c_1, c_2, \dots, c_l$ ，这些整数均为 0 或 1 。

当 $M = 0$ 时，输入中没有模式行。

【输出格式】

输出到 *pattern.out* 文件。

输出 $G - 2$ 行，依次对应起始符号 $2, 3, \dots, G - 1$ 。

若所有完成串都会触发警报，输出：

```
1 YES
```

否则输出：

```
1 NO L
```

保证需要输出的有限 $L < 2^{63}$ 。

【样例 1】

输入

```
1 6 6 2
2 2 1 0
3 2 1 1
4 3 2 2 2
5 4 2 3 0
6 4 2 1 3
7 5 1 5
8 2 0 0
9 2 1 1
```

输出

【样例 1 输出】

```
1 NO 1
2 NO 2
3 NO 3
4 YES
```

说明

符号 2 可以生成长度为 1 的安全串；符号 3 可以生成 01；符号 4 可以生成 010，所以前三行的最短长度依次为 1, 2, 3。符号 5 只能展开为自身，没有完成串，因此最后一行输出 YES。

【样例 2】

输入

```
1 4 3 1
2 2 1 0
3 2 1 1
4 3 2 2 2
5 2 0 0
```

输出

【样例 2 输出】

```
1 NO 1
2 NO 2
```

说明

符号 2 可以生成长度为 1 的 0 或 1，二者都不包含模式 00。符号 3 可以生成 01，所以它的最短安全完成串长度为 2。

【样例 3】

输入

```
1 3 1 0
2 2 1 2
```

输出

【样例 3 输出】

```
1 YES
```

说明

符号 2 只能展开为自身，不存在完成串，因此输出 YES。

【样例 4】

输入与输出见 sample 目录下的 *pattern/pattern4.in* 和 *pattern/pattern4.ans*。

【数据范围与子任务】

对于所有数据：

- $3 \leq G \leq 102$, $G - 2 \leq N \leq 100$, $0 \leq M \leq 50$;
- $2 \leq a < G$, 每个 $2, 3, \dots, G - 1$ 都至少在一条规则中作为左端出现;
- $k \geq 1$, $0 \leq b_i < G$, 所有规则的 k 之和不超过 100;
- $l \geq 1$, $c_i \in \{0, 1\}$, 所有模式的 l 之和不超过 50。

本题采用子任务捆绑计分：同一子任务内的全部测试点均通过后才能获得该子任务分数。

子任务	分值	限制
1	15	$M = 0$
2	15	$N = G - 2$
3	30	所有模式的长度之和不超过 10
4	40	无额外限制